
rxnet — Guía de usuario: C

FSM y Redes de Petri sobre un runtime síncrono reactivo

2026-05-10

Contents

rxnet — Guía de usuario	5
1. El problema que resuelve rxnet	5
2. La hipótesis síncrona	5
3. El tick y sus fases	6
Por qué esa separación de fases	6
4. Máquinas de estado finito (FSM)	7
4.1 Cuándo usar una FSM	7
4.2 Anatomía de una máquina	7
4.3 Guards y acciones	8
4.4 Callbacks de fase	8
4.5 Ejemplo completo: luz con auto-apagado	9
4.6 El patrón de señales de entrada en FSM	10
5. Redes de Petri (PN)	11
5.1 Cuándo usar una red de Petri	11
5.2 Conceptos fundamentales	11
5.3 Semántica greedy-sequential	12
5.4 Anatomía de una red	12
5.5 Lugares de señal (signal places)	13
5.6 Ejemplo completo: luz toggle	14
6. Modelos de ejecución concurrente	15
6.1 Ejecutivo cíclico — rx_cyclic_exec	15
6.2 Multitarea cooperativa — rx_coop_exec	16
6.3 Threads paralelos — rx_thread_exec	16
6.4 Resumen comparativo de executors	17
6.5 Comunicación entre nodos	17
7. Cuándo usar FSM, cuándo PN	18
Usa FSM cuando...	18
Usa PN cuando...	18
Mezclar ambos (patrón habitual)	19

8. Referencia rápida de la API	19
Core runtime	19
FSM	20
Petri Net	21
Firmas de callbacks	22
9. Depuración y trazado	23
Activar en tiempo de compilación	23
Uso básico	23
Trazado de fases	24
Eventos de usuario	25
Visualizar desde el Mac de desarrollo	25
Hooks de plataforma	25
Lectura adicional	25

List of Figures

rxnet — Guía de usuario

1. El problema que resuelve rxnet

Los sistemas reactivos responden a eventos del entorno: pulsaciones de botón, lecturas de sensor, mensajes de red, expiración de temporizadores. El reto no es *qué hacer* ante un evento, sino *cuándo hacerlo* y *qué hay que ver* en ese momento.

Sin disciplina, el código reactivo acumula dos tipos de bugs difíciles de reproducir:

- **Race conditions de lectura:** el guard de una transición lee un bit de hardware a mitad de ciclo, cuando otro módulo ya lo consumió.
- **Efectos secundarios prematuros:** una acción ejecutada durante la evaluación modifica estado que otro módulo todavía no evaluó.

rxnet impone una disciplina que los elimina por construcción: **toda la información que entra en el sistema se lee exactamente una vez por ciclo, al principio, y todos los efectos secundarios se ejecutan exactamente una vez, al final.**

2. La hipótesis síncrona

rxnet está inspirada en los *lenguajes síncronos* (Esterel, Lustre, Signal) y en sus derivados industriales (SCADE, Simulink). La idea central se llama la **hipótesis síncrona**:

La computación de un tick es instantánea. El mundo externo no cambia mientras el sistema está procesando.

Esto es una abstracción, no una afirmación física. En la práctica significa:

- El tick debe completarse antes de que llegue el siguiente evento significativo.
- Si el tick tarda más que el período de muestreo, el sistema tiene un problema de diseño, no de concurrencia.

Bajo la hipótesis síncrona, **no hay carrera de datos posible dentro del tick**: todos los módulos leen el mismo snapshot de entradas, y nadie ve los cambios de los demás hasta el siguiente tick.

3. El tick y sus fases

Cada llamada a `rx_tick()` ejecuta cuatro fases de nodo en orden estricto:

#	Fase	Qué hace cada nodo
1	Latch	<code>latch_inputs_cb(ctx, user)</code> — leer GPIO, calcular señales derivadas, tomar snapshot
2	Evaluate	<code>evaluate()</code> — decidir siguiente estado / flags (solo lectura del snapshot)
3	Commit	<code>commit()</code> — aplicar decisiones; tras el commit de todos, ejecutar acciones diferidas
4	Dump	<code>dump_outputs_cb(ctx, user)</code> — escribir GPIO, actualizar salidas

Por qué esa separación de fases

La separación **evaluate / commit / dump** no es arbitraria:

- **Evaluate** solo puede leer, nunca escribir estado observable. Todos los módulos ven el mismo estado del sistema al mismo tiempo.
- **Commit** aplica las decisiones. Las acciones diferidas encoladas durante `evaluate` se ejecutan después de que todos los nodos activos hayan hecho commit, antes de cualquier `dump`. Una acción ve el estado comprometido de todos los módulos, no solo el propio.
- **Dump** publica las salidas después de commit y de las acciones diferidas.

La separación **latch / dump** garantiza que las lecturas de hardware ocurren *antes* de evaluar (snapshot limpio) y las escrituras *después* de decidir (salida consistente).

4. Máquinas de estado finito (FSM)

4.1 Cuándo usar una FSM

Una FSM es la herramienta adecuada cuando el comportamiento del módulo depende de su **historia**: no solo del evento actual sino de lo que pasó antes.

Ejemplos típicos:

Problema	Estados naturales
Luz con botón	OFF, ON
Semáforo	ROJO, VERDE, AMARILLO
Puerta con cerrojo	ABIERTA, CERRADA, BLOQUEADA
Protocolo de comunicación	IDLE, CONECTANDO, CONECTADO, ERROR
Menú de display	MENU_PRINCIPAL, SUBMENU_A, SUBMENU_B

4.2 Anatomía de una máquina

Una `rx_fsm_machine` se define completamente en tiempo de compilación: transiciones, estados y callbacks son punteros a funciones estáticas. No hay allocación dinámica.

```
1 #include "rxnet/fsm.h"
2
3 /* Estados: enteros arbitrarios */
4 enum { STATE_OFF = 0, STATE_ON = 1 };
5
6 /* Datos de usuario pasados a todos los callbacks */
7 typedef struct {
8     bool latched_event;
9     int output_enabled;
10 } light_data_t;
11
12 /* Tabla de transiciones (puede ser static const) */
13 static const rx_fsm_transition transitions[] = {
14     /* { from_state, to_state, guard, action } */
15     { STATE_OFF, STATE_ON, button_pressed, light_on },
16     { STATE_ON, STATE_OFF, button_pressed, light_off },
17 };
```

```

18
19 static light_data_t data;
20 static rx_fsm_machine machine;
21
22 rx_fsm_machine_init(
23     &machine,
24     "light",                /* nombre (debug) */
25     STATE_OFF,              /* estado inicial */
26     transitions,
27     sizeof(transitions) / sizeof(*transitions),
28     &data,                  /* user pointer */
29     light_latch_cb,         /* fase latch */
30     light_dump_cb           /* fase dump */
31 );

```

Semántica first-match: en cada tick, el runtime recorre las transiciones en orden de declaración y ejecuta la *primera* cuya guardia devuelva distinto de cero y cuyo estado origen coincida con el estado actual.

4.3 Guards y acciones

```

1  /* Guard: pura, sin efectos secundarios; devuelve != 0 para "verdadero"
   */
2  static int button_pressed(const rx_fsm_context *ctx, void *user) {
3      const light_data_t *d = user;
4      (void)ctx;
5      return d->latched_event;
6  }
7
8  /* Acción: deferred, corre tras commit con estado ya comprometido */
9  static void light_on(rx_fsm_context *ctx, void *user) {
10     light_data_t *d = user;
11     (void)ctx;
12     d->output_enabled = 1;
13 }
14
15 static void light_off(rx_fsm_context *ctx, void *user) {
16     light_data_t *d = user;
17     (void)ctx;
18     d->output_enabled = 0;
19 }

```

Regla importante: los guards nunca deben tener efectos secundarios. Son funciones puras de lectura. Los efectos van en las acciones.

4.4 Callbacks de fase

```
1 /* Fase latch: tomar snapshot de entradas */
2 static void lightLatchCb(rx_fsm_context *ctx, void *user) {
3     light_data_t *d = user;
4     (void)ctx;
5     d->latched_event = read_button_gpio();
6 }
7
8 /* Fase dump: escribir salidas */
9 static void lightDumpCb(rx_fsm_context *ctx, void *user) {
10     const light_data_t *d = user;
11     (void)ctx;
12     set_led_gpio(d->output_enabled);
13 }
```

4.5 Ejemplo completo: luz con auto-apagado

```
1 #include <stdint.h>
2 #include <stdbool.h>
3 #include "rxnet/fsm.h"
4
5 enum { STATE_OFF = 0, STATE_ON = 1 };
6
7 typedef struct {
8     bool    latched_event;
9     bool    enabled;
10    uint32_t timeout_ms;
11    uint32_t deadline_ms;
12    uint32_t now_ms;
13 } auto_light_data_t;
14
15 static int pressed(const rx_fsm_context *ctx, void *user) {
16     (void)ctx;
17     return ((auto_light_data_t *)user)->latched_event;
18 }
19
20 static int timed_out(const rx_fsm_context *ctx, void *user) {
21     const auto_light_data_t *d = user;
22     (void)ctx;
23     return d->now_ms >= d->deadline_ms;
24 }
25
26 static void turn_on(rx_fsm_context *ctx, void *user) {
27     auto_light_data_t *d = user;
28     (void)ctx;
29     d->enabled = true;
30     d->deadline_ms = d->now_ms + d->timeout_ms;
31 }
32
```

```

33 static void turn_off(rx_fsm_context *ctx, void *user) {
34     auto_light_data_t *d = user;
35     (void)ctx;
36     d->enabled = false;
37 }
38
39 static void latch_cb(rx_fsm_context *ctx, void *user) {
40     auto_light_data_t *d = user;
41     (void)ctx;
42     d->now_ms = get_time_ms();
43     d->latched_event = read_button_gpio();
44 }
45
46 static void dump_cb(rx_fsm_context *ctx, void *user) {
47     const auto_light_data_t *d = user;
48     (void)ctx;
49     set_led_gpio(d->enabled);
50 }
51
52 static const rx_fsm_transition transitions[] = {
53     { STATE_OFF, STATE_ON, pressed, turn_on },
54     { STATE_ON, STATE_ON, pressed, turn_on }, /* reset timer */
55     { STATE_ON, STATE_OFF, timed_out, turn_off },
56 };
57
58 /* Montaje */
59 static auto_light_data_t data = { .timeout_ms = 5000 };
60 static rx_fsm_machine machine;
61
62 void light_init(void) {
63     rx_fsm_machine_init(
64         &machine, "auto_light", STATE_OFF,
65         transitions, sizeof(transitions) / sizeof(*transitions),
66         &data, latch_cb, dump_cb
67     );
68 }

```

4.6 El patrón de señales de entrada en FSM

La forma canónica de conectar hardware a una FSM en rxnet:

Hardware	<code>latch_inputs_cb</code>	guards / actions
GPIO → evento	<code>data->latched = read_gpio()</code>	<code>if (data->latched)...</code>
(vivo, volátil)	(snapshot estable)	(solo leen snapshot)

El callback `latch_inputs_cb` es el único punto de contacto con el hardware. El resto de la máquina trabaja con copias estables.

5. Redes de Petri (PN)

5.1 Cuándo usar una red de Petri

Una red de Petri es la herramienta adecuada cuando hay **conurrencia natural**: varios procesos que avanzan en paralelo y necesitan sincronizarse.

Ejemplos típicos:

Problema	Modela bien porque...
Botón toggle	Un token fluye entre P_OFF y P_ON
Buffer productor-consumidor	Tokens representan slots llenos/vacíos
Semáforo / mutex	Un token en P_LIBRE controla el acceso
Protocolo de handshake	Ambos lados avanzan y se sincronizan
Recursos compartidos	Tokens cuentan instancias disponibles

La PN no reemplaza a la FSM: cuando el comportamiento es esencialmente lineal (un agente con historia), la FSM es más clara. Cuando hay múltiples flujos que se sincronizan, la PN es más clara.

5.2 Conceptos fundamentales

Concepto	Símbolo	Significado
Lugar (place)	○	condición, estado, recurso, mensaje pendiente
Token	•	unidad de recurso, condición activa
Transición	rect.	evento, paso de proceso

Concepto	Símbolo	Significado
Arco de entrada	$\rightarrow T$	condición necesaria para disparar (consume tokens)
Arco de salida	$T \rightarrow$	condición que produce al disparar (produce tokens)

Una transición **está habilitada** cuando todos sus lugares de entrada tienen suficientes tokens ($\geq$ peso del arco). Una transición habilitada **dispara**: consume los tokens de sus entradas y produce tokens en sus salidas.

5.3 Semántica greedy-sequential

rxnet usa evaluación **greedy-sequential**: las transiciones se evalúan en orden de declaración, y las anteriores *consumen tokens inmediatamente*, antes de que las posteriores se evalúen.

Consecuencia práctica: **el orden de declaración implica prioridad**. Si dos transiciones compiten por el mismo token, gana la que se declara primero.

```

1  static const rx_pn_arc consume_x[] = {{ .place_id = P_X, .weight = 1
    }};
2  static const rx_pn_arc produce_a[] = {{ .place_id = P_A, .weight = 1
    }};
3  static const rx_pn_arc produce_b[] = {{ .place_id = P_B, .weight = 1
    }};
4
5  static const rx_pn_transition transitions[] = {
6      /* T0 declarada antes → prioridad sobre T1 */
7      { consume_x, 1, produce_a, 1, NULL, NULL },
8      /* T1: no dispara en el mismo tick si T0 consumió el token de P_X
        */
9      { consume_x, 1, produce_b, 1, NULL, NULL },
10 };

```

5.4 Anatomía de una red

```

1  #include "rxnet/pn.h"
2
3  enum { P_OFF = 0, P_ON = 1, P_REQUEST = 2 };
4
5  static const rx_pn_arc arcs_off_req[] = {{ P_OFF, 1 }, { P_REQUEST, 1
    }};

```

```

6 static const rx_pn_arc arcs_on[] = {{ P_ON, 1 }};
7 static const rx_pn_arc arcs_on_req[] = {{ P_ON, 1 }, { P_REQUEST, 1
    }};
8 static const rx_pn_arc arcs_off[] = {{ P_OFF, 1 }};
9
10 static const rx_pn_transition transitions[] = {
11     /* off + request → on */
12     { arcs_off_req, 2, arcs_on, 1, NULL, NULL },
13     /* on + request → off */
14     { arcs_on_req, 2, arcs_off, 1, NULL, NULL },
15 };
16
17 static const int initial_places[] = { 1, 0, 0 }; /* token en P_OFF */
18 static rx_pn_net net;
19
20 rx_pn_net_init(
21     &net,
22     "light",
23     initial_places, 3,
24     transitions, 2,
25     NULL, /* user pointer */
26     light_latch_cb, /* fase latch — NULL → noop */
27     light_dump_cb /* fase dump — NULL → noop */
28 );

```

Los arrays de arcos y transiciones pueden ser **static const**: el runtime no los modifica, solo los lee. Los tokens (`net.places[]`) sí se modifican en cada tick.

5.5 Lugares de señal (signal places)

Un **lugar de señal** no acumula tokens: se *restablece* en cada latch a 0 o 1 según una condición del entorno.

```

1 /* En latch_inputs_cb: reescribir P_TOGGLE_DUE en cada tick */
2 static void blink_latch_cb(rx_pn_context *ctx, void *user) {
3     rx_pn_net *net = user;
4     bool is_blinking = net->places[P_X1] > 0 || net->places[P_X2] > 0;
5     net->places[P_TOGGLE_DUE] = (is_blinking && timer_elapsed()) ? 1 :
6         0;
7     (void)ctx;
8 }

```

Contrasta con los **lugares de cola** (como P_REQUEST), que acumulan tokens:

```

1 /* En latch_inputs_cb: añadir un token si hay pulsación */
2 static void light_latch_cb(rx_pn_context *ctx, void *user) {
3     rx_pn_net *net = user;
4     if (read_button_gpio())

```

```

5         net->places[P_REQUEST]++; /* acumula; se consume 1 por tick
        */
6     (void)ctx;
7 }

```

Tipo de lugar	Comportamiento	Ejemplo
Señal	Se reescribe a 0/1 cada latch	Timer expirado, sensor activo
Cola	Acumula; transición consume 1	Pulsación de botón pendiente
Estado	Token único que fluye	ON/OFF, ROJO/VERDE
Contador	N tokens = N recursos	Slots libres en buffer

5.6 Ejemplo completo: luz toggle

```

1  #include <stdbool.h>
2  #include "rxnet/pn.h"
3
4  enum { P_OFF = 0, P_ON = 1, P_REQUEST = 2 };
5
6  typedef struct {
7      bool output;
8  } light_pn_data_t;
9
10 static void action_turn_on(rx_pn_context *ctx, void *user) {
11     ((light_pn_data_t *)user)->output = true; (void)ctx;
12 }
13
14 static void action_turn_off(rx_pn_context *ctx, void *user) {
15     ((light_pn_data_t *)user)->output = false; (void)ctx;
16 }
17
18 static void latch_cb(rx_pn_context *ctx, void *user) {
19     rx_pn_net *net = user;
20     if (read_button_gpio())
21         net->places[P_REQUEST]++;
22     (void)ctx;
23 }
24
25 static void dump_cb(rx_pn_context *ctx, void *user) {
26     set_led_gpio(((light_pn_data_t *)user)->output); (void)ctx;
27 }
28
29 static const rx_pn_arc arcs_off_req[] = {{ P_OFF, 1 }, { P_REQUEST, 1
30     }};
31 static const rx_pn_arc arcs_on[] = {{ P_ON, 1 }};

```

```

31 static const rx_pn_arc arcs_on_req[] = {{ P_ON, 1 }, { P_REQUEST, 1
    }};
32 static const rx_pn_arc arcs_off[] = {{ P_OFF, 1 }};
33
34 static const rx_pn_transition transitions[] = {
35     { arcs_off_req, 2, arcs_on, 1, NULL, action_turn_on },
36     { arcs_on_req, 2, arcs_off, 1, NULL, action_turn_off },
37 };
38
39 static const int initial[] = { 1, 0, 0 };
40 static light_pn_data_t pn_data;
41 static rx_pn_net net;
42
43 void light_pn_init(void) {
44     pn_data.output = false;
45     rx_pn_net_init(
46         &net, "light",
47         initial, 3,
48         transitions, 2,
49         &net, /* user = net para acceder a places[] en latch_cb */
50         latch_cb, dump_cb
51     );
52 }

```

6. Modelos de ejecución concurrente

Esta sección explica cómo rxnet se relaciona con los tres modelos clásicos de ejecución en sistemas embebidos y de tiempo real.

rxnet incluye tres **executors** que gestionan el timing y el scheduling automáticamente. El período de cada runtime se lee de `rt->period_us`, que el propio runtime calcula como el MCD de los períodos de sus nodos.

6.1 Ejecutivo cíclico — `rx_cyclic_exec`

Tabla de despacho estática con hiperperíodo. Un único hilo, orden de slots determinista. Adecuado para bare-metal y configuraciones RTOS simples.

```

1 #include "rxnet/cyclic.h"
2
3 rx_cyclic_exec ce;
4 rx_cyclic_exec_init(&ce);

```

```

5 rx_cyclic_exec_add(&ce, &fast_rt.runtime); /* período 10 ms */
6 rx_cyclic_exec_add(&ce, &slow_rt.runtime); /* período 20 ms */
7 rx_cyclic_exec_run(&ce); /* retorna tras rx_cyclic_exec_stop(&ce) */

```

El ejecutor calcula automáticamente: - **base** = $\text{MCD}(10, 20) = 10 \text{ ms}$ → período base - **hyper** = $\text{MCM}(10, 20) = 20 \text{ ms}$ → 2 slots - Slot 0: **fast_rt** + **slow_rt**; Slot 1: solo **fast_rt**

Cuándo usarlo: periodos fijos conocidos en compilación, determinismo máximo.

Limitaciones: todos los períodos deben ser múltiplos enteros del MCD.

6.2 Multitarea cooperativa — rx_coop_exec

Scheduling dinámico por deadline. Un único hilo comprueba qué runtime tiene el deadline más próximo y lo ejecuta. No requiere que los períodos sean múltiplos entre sí.

```

1 #include "rxnet/coop.h"
2
3 rx_coop_exec ce;
4 rx_coop_exec_init(&ce);
5 rx_coop_exec_add(&ce, &rt_a.runtime); /* período 10 ms */
6 rx_coop_exec_add(&ce, &rt_b.runtime); /* período 15 ms */
7 rx_coop_exec_run(&ce); /* retorna tras rx_coop_exec_stop(&ce) */

```

El ejecutor avanza cada deadline desde su último disparo (no desde “ahora”), evitando la acumulación de deriva de fase incluso cuando un runtime se retrasa.

Cuándo usarlo: períodos irregulares, tareas que a veces se alargan un poco, sin overhead de threads.

Limitaciones: una tarea muy lenta puede retrasar a todas las demás.

6.3 Threads paralelos — rx_thread_exec

Un thread por nodo periódico. El ejecutor crea grupos de activación para cada instante absoluto activo, sin materializar una tabla de hiperperíodo. Dos barreras BSP por grupo sincronizan las fases reactivas con verdadero paralelismo:

- **eval_b**: todos los nodos activos han hecho **latch** y **evaluate**.
- **commit_b**: todos los nodos activos han hecho **commit** y despachado acciones diferidas.

El último nodo del último runtime corre en el hilo llamante (útil para el nodo CLI/stdin). Los nodos async (**period_us** = 0) no están soportados en **rx_thread_exec**, porque un solo thread async no puede participar con seguridad en varios grupos de activación solapados.

```

1 #include "rxnet/thread.h"
2
3 rx_thread_exec te;
4 rx_thread_exec_init(&te);
5 rx_thread_exec_add(&te, &pn_rt.runtime); /* nodos PN → threads */
6 rx_thread_exec_add(&te, &cli_rt.runtime); /* CLI → hilo principal */
7 rx_thread_exec_run(&te); /* retorna tras rx_thread_exec_stop(&te) */

```

Todos los executors ofrecen un hook `rx_*_exec_on_stop()` para ejecutar una operación de cierre de la aplicación/modelo una vez antes de que `run()` retorne.

Cuándo usarlo: varios nodos con trabajo de cómputo intensivo que se benefician del paralelismo real; sistemas con múltiples cores.

Limitaciones: overhead de sincronización de barreras; requiere `-lpthread` en POSIX (o el soporte de tasks equivalente en FreeRTOS/Zephyr).

6.4 Resumen comparativo de executors

	<code>rx_cyclic_exec</code>	<code>rx_coop_exec</code>	<code>rx_thread_exec</code>
Threads	1	1	1 por nodo
Dispatch	Tabla estática (hiperperíodo del executive)	Próxima activación + deadline	Grupos de activación + barreras BSP
Períodos	Múltiplos del MCD	Cualquiera	Cualquiera
Paralelismo	No	No	Sí
Overhead	Mínimo	Mínimo	Barreras mutex
Casos típicos	Bare-metal, RTOS simple	Períodos irregulares	Múltiples cores

6.5 Comunicación entre nodos

Estado de FSM como señal: un nodo puede leer `machine_a.state` directamente en sus guards (el estado fue comprometido en commit, antes de que cualquier nodo ejecute el siguiente tick).

Tokens de PN como mensajes: el productor escribe `net.places[P_BUFFER]` en su `latch_inputs_cb`; el consumidor dispara la transición que consume ese token.

Acciones diferidas: cualquier callback puede encolar una acción para el despacho deferred del tick actual:

```
1 rx_context_enqueue_deferred_action(ctx, my_action_fn, user_ptr);
```

Las acciones se ordenan por prioridad (FIFO dentro del mismo nivel):

```
1 rx_context_enqueue_deferred_action_p(ctx, send_alarm, user,
   RX_PRIORITY_CRITICAL);
2 rx_context_enqueue_deferred_action_p(ctx, log_event, user,
   RX_PRIORITY_NORMAL);
3 rx_context_enqueue_deferred_action_p(ctx, update_stats, user,
   RX_PRIORITY_LOW);
```

Nota sobre thread safety: `rx_tick()` no es thread-safe por sí mismo. Con `rx_thread_exec`, las barreras garantizan la consistencia del snapshot; con los otros executors (hilo único) no se necesita sincronización adicional.

7. Cuándo usar FSM, cuándo PN

Usa FSM cuando...

- El módulo tiene **un agente con historia**: hay un único “estado del módulo” en cada momento.
- Las transiciones son mutuamente excluyentes: estar en un estado excluye estar en otro.
- El control de flujo es lineal (incluso si tiene bucles y ramas).
- Necesitas guards complejos que leen múltiples variables.

Ejemplos:

- Semáforo: ROJO → VERDE → AMARILLO → ROJO
- Cerrojo: DESBLOQUEADO → BLOQUEADO
- Protocolo: IDLE → SYN_SENT → ESTABLISHED

Usa PN cuando...

- Hay **múltiples agentes independientes** que se sincronizan.
- Los recursos son contables (N slots, M conexiones disponibles).
- El paralelismo es natural: varias actividades ocurren al mismo tiempo.
- Necesitas modelar producción/consumo de forma explícita.

Ejemplos:

- Productor/consumidor: places `P_LLENOS` + `P_VACIOS`
- Semáforo binario: `P_MUTEX` (0 o 1 token)
- Rendezvous: A espera a B, B espera a A

Mezclar ambos (patrón habitual)

Lo más potente es combinarlos en un único runtime base. La FSM describe la **política** (qué modo estoy); la PN describe los **recursos y sincronización** (qué tengo disponible):

```
1  /* Sistema de acceso a sala:
2   * FSM: gestiona el ciclo de apertura de puerta
3   * PN:  gestiona los recursos (tokens = personas dentro) */
4
5  rx_context      ctx;
6  rx_runtime      rt;
7  rx_fsm_machine  door_machine;
8  rx_pn_net       capacity_net;
9
10 rx_context_init(&ctx);
11 rx_runtime_init(&rt, &ctx, 2);
12
13 door_fsm_init(&door_machine);
14 capacity_pn_init(&capacity_net);
15
16 rx_runtime_add_node(&rt, &door_machine.node, 0, 0);
17 rx_runtime_add_node(&rt, &capacity_net.node, 0, 0);
18
19 for (;;) {
20     rx_tick(&rt);    /* FSM y PN avanzan juntos en el mismo tick */
21     sleep_10ms();
22 }
```

Al compartir el mismo `rx_context`, las acciones diferidas de la FSM y de la PN se ejecutan todas en el mismo despacho deferred del tick, después de commit y antes de dump.

8. Referencia rápida de la API

Core runtime

```

1  #include "rxnet/runtime.h"
2
3  /* Contexto: cola de acciones diferidas */
4  rx_context ctx;
5  rx_context_init(&ctx);
6
7  /* Runtime: lista de nodos */
8  rx_runtime rt;
9  rx_runtime_init(&rt, &ctx, node_capacity);
10
11 /* Registrar nodos */
12 rx_runtime_add_node(&rt, &machine.node, period_us, deadline_us); /*
    FSM o PN */
13
14 /* Ejecutar un ciclo */
15 rx_tick(&rt);
16
17 /* Encolar acción diferida con prioridad por defecto (NORMAL) */
18 rx_context_enqueue_deferred_action(&ctx, action_fn, user_ptr);
19
20 /* Encolar acción diferida con prioridad explícita */
21 rx_context_enqueue_deferred_action_p(&ctx, action_fn, user_ptr,
    RX_PRIORITY_HIGH);
22
23 /* Prioridades disponibles */
24 /* RX_PRIORITY_CRITICAL = 3 alarmas, paradas de emergencia */
25 /* RX_PRIORITY_HIGH     = 2 control en tiempo real */
26 /* RX_PRIORITY_NORMAL   = 1 lógica de aplicación (defecto) */
27 /* RX_PRIORITY_LOW      = 0 telemetría, logging */
28
29 /* Worker pool asíncrono (implementación provista por el usuario) */
30 rx_runtime_set_worker_pool(&rt, &my_pool.base); /* activar */
31 rx_runtime_set_worker_pool(&rt, NULL); /* volver a modo sí
    ncrono */
32
33 /* Vtable que el usuario debe implementar */
34 struct rx_worker_pool {
35     void (*post)(rx_worker_pool *self,
36                 rx_deferred_action_fn fn,
37                 rx_context *ctx,
38                 void *user,
39                 rx_priority_t priority);
40 };

```

FSM

```

1  #include "rxnet/fsm.h"
2

```

```

3  /* Tipos de función */
4  typedef int  (*rx_fsm_guard_fn)(const rx_fsm_context *ctx, void *user);
5  typedef void (*rx_fsm_action_fn)(rx_fsm_context *ctx, void *user);
6  typedef void (*rx_fsm_node_phase_fn)(rx_fsm_context *ctx, void *user);
7
8  /* Transición */
9  struct rx_fsm_transition {
10     int      from_state;
11     int      to_state;
12     rx_fsm_guard_fn  guard; /* NULL → siempre dispara */
13     rx_fsm_action_fn action; /* NULL → sin efecto */
14 };
15
16 /* Inicializar máquina (stack allocation) */
17 void rx_fsm_machine_init(
18     rx_fsm_machine *machine,
19     const char *name,
20     int initial_state,
21     const rx_fsm_transition *transitions,
22     size_t transition_count,
23     void *user,
24     rx_fsm_node_phase_fn latch_inputs, /* NULL → noop */
25     rx_fsm_node_phase_fn dump_outputs /* NULL → noop */
26 );
27
28 /* Runtime dedicado FSM (contiene contexto propio) */
29 rx_fsm_runtime fsm_rt;
30 rx_fsm_runtime_init(&fsm_rt, machine_capacity);
31 rx_fsm_runtime_add_machine(&fsm_rt, &machine, period_us, deadline_us);
32 rx_fsm_tick(&fsm_rt);

```

Petri Net

```

1  #include "rxnet/pn.h"
2
3  /* Tipos de función */
4  typedef int  (*rx_pn_guard_fn)(const rx_pn_context *ctx, void *user);
5  typedef void (*rx_pn_action_fn)(rx_pn_context *ctx, void *user);
6  typedef void (*rx_pn_node_phase_fn)(rx_pn_context *ctx, void *user);
7
8  /* Arco */
9  struct rx_pn_arc {
10     size_t place_id;
11     int weight;
12 };
13
14 /* Transición */
15 struct rx_pn_transition {
16     const rx_pn_arc *consume; /* array de arcos de entrada */

```

```

17     size_t          consume_count;
18     const rx_pn_arc *produce;      /* array de arcos de salida */
19     size_t          produce_count;
20     rx_pn_guard_fn  guard;       /* NULL → siempre dispara si
                                     habilitada */
21     rx_pn_action_fn action;      /* deferred, NULL → noop */
22 };
23
24 /* Inicializar red (stack allocation) */
25 int rx_pn_net_init(
26     rx_pn_net          *net,
27     const char          *name,
28     const int          *initial_places,
29     size_t             place_count,
30     const rx_pn_transition *transitions,
31     size_t             transition_count,
32     void               *user,
33     rx_pn_node_phase_fn latch_inputs, /* NULL → noop */
34     rx_pn_node_phase_fn dump_outputs /* NULL → noop */
35 );
36
37 /* Runtime dedicado PN */
38 rx_pn_runtime pn_rt;
39 rx_pn_runtime_init(&pn_rt, net_capacity);
40 rx_pn_runtime_add_net(&pn_rt, &net, period_us, deadline_us);
41 rx_pn_tick(&pn_rt);

```

Firmas de callbacks

```

1  /* Guard: pura, sin efectos secundarios; devuelve != 0 para verdadero
   */
2  static int my_guard(const rx_fsm_context *ctx, void *user) {
3      (void)ctx;
4      return ((my_data_t *)user)->latched_event;
5  }
6
7  /* Acción: deferred, corre tras commit con estado ya comprometido */
8  static void my_action(rx_fsm_context *ctx, void *user) {
9      (void)ctx;
10     ((my_data_t *)user)->output = 1;
11 }
12
13 /* Latch: snapshot hardware, señales derivadas */
14 static void latch_cb(rx_fsm_context *ctx, void *user) {
15     (void)ctx;
16     my_data_t *d = user;
17     d->latched_event = read_gpio(d->button_pin);
18 }
19

```

```
20 /* Dump: escribir salidas al hardware */
21 static void dump_cb(rx_fsm_context *ctx, void *user) {
22     (void)ctx;
23     const my_data_t *d = user;
24     write_gpio(d->led_pin, d->output);
25 }
```

9. Depuración y trazado

rxnet incluye un subsistema de trazado **completamente opcional**: cuando no se activa, genera **cero código y cero datos**. Todos los macros se expanden a `((void)0)`, los campos extra de los nodos no existen, y el enlazador no incluye ningún símbolo relacionado.

Activar en tiempo de compilación

```
1 CFLAGS += -DRX_TRACE_ENABLE
```

Con este flag, `rx_node` gana dos campos (`trace` y `trace_nid`), y el runtime registra eventos `NODE_START/END`, transiciones FSM y disparos PN en un buffer circular de capacidad fija (sin heap).

Uso básico

```
1 #include "rxnet/trace.h"
2
3 rx_trace_buf_t tracer;
4
5 /* 1. Inicializar el buffer (phases=0: sin eventos de fase; =1: con
   ellos) */
6 rx_trace_init(&tracer, 0);
7
8 /* 2. Adjuntar a cada nodo que quieras trazar */
9 rx_trace_attach(&tracer, &light_machine.node, 0); /* nid = 0 */
10 rx_trace_attach(&tracer, &blink_machine.node, 1); /* nid = 1 */
11
12 /* 3. Registrar nombres (opcional, para el informe HTML) */
13 rx_trace_set_node_name(&tracer, 0, "light");
14 rx_trace_set_state_name(&tracer, 0, STATE_OFF, "OFF");
15 rx_trace_set_state_name(&tracer, 0, STATE_ON, "ON");
16
```

```

17 rx_trace_set_node_name (&tracer, 1, "blink");
18 rx_trace_set_state_name(&tracer, 1, BLINK_SLOW, "SLOW");
19 rx_trace_set_state_name(&tracer, 1, BLINK_FAST, "FAST");
20
21 /* 4. Ejecutar el sistema normalmente... */
22
23 /* 5. Exportar a fichero binario */
24 rx_trace_export(&tracer, "trace.rxnt");

```

`rx_trace_attach()` sigue siendo útil cuando quieres controlar manualmente el `nid` de cada nodo. Para la mayoría de los casos, sin embargo, ahora existe una opción más cómoda a nivel de runtime:

```

1 rx_trace_init(&tracer, 0);
2
3 /* Adjunta todos los nodos registrados en rt.runtime */
4 rx_trace_attach_runtime(&tracer, &rt.runtime);

```

`rx_trace_attach_runtime()` es **idempotente e incremental**: si se invoca dos veces sobre el mismo runtime no reenumera ni vuelve a adjuntar los nodos ya conocidos, y si entre llamadas se añaden nuevos nodos al runtime, una nueva invocación asigna `trace_nid` solo a esos nodos nuevos.

Ejemplo:

```

1 rx_trace_attach_runtime(&tracer, &rt.runtime);
2
3 /* más tarde: se amplía la red */
4 rx_fsm_runtime_add_machine(&rt, &aux_machine, 0, 0);
5 rx_runtime_build(&rt.runtime);
6
7 rx_trace_attach_runtime(&tracer, &rt.runtime); /* solo adjunta
   aux_machine */

```

Para redes de Petri, registrar también lugares y transiciones:

```

1 rx_trace_set_node_name (&tracer, 2, "blink_pn");
2 rx_trace_set_place_name(&tracer, 2, P_OFF, "OFF");
3 rx_trace_set_place_name(&tracer, 2, P_ON, "ON");
4 rx_trace_set_trans_name(&tracer, 2, T_TOGGLE, "toggle");

```

Trazado de fases

Pasar `phases=1` a `rx_trace_init` registra el inicio y fin de cada fase (latch / eval / commit / dump), permitiendo medir cuánto tarda cada una:

```

1 rx_trace_init(&tracer, 1); /* phases activadas */

```

Eventos de usuario

Desde cualquier parte del código:

```
1 /* lid = label id (0..RX_TRACE_MAX_LABELS-1); value = uint16_t */
2 rx_trace_user(&tracer, 0, 42);
3
4 /* Registrar el nombre del label */
5 rx_trace_set_label_name(&tracer, 0, "temperatura");
```

Visualizar desde el Mac de desarrollo

Una vez que tienes el fichero `.rxnt`, ejecuta el decodificador Python:

```
1 python -m rxnet.tools.trace trace.rxnt --report trace.html --open
2 python -m rxnet.tools.trace trace.rxnt --stats                # WCRT por nodo
   en texto
3 python -m rxnet.tools.trace trace.rxnt --perfetto out.json
```

El informe HTML contiene diagramas de cada FSM/PN y una tabla WCRT. El fichero Perfetto se puede abrir en ui.perfetto.dev para ver la línea de tiempo interactiva.

Hooks de plataforma

Por defecto el trazado usa el reloj y el mutex de `rxnet/port.h` (POSIX, FreeRTOS o Zephyr según la plataforma destino). Se pueden sobrescribir antes de incluir `trace.h`:

```
1 #define RX_TRACE_NOW_NS()      my_platform_clock_ns()
2 #define RX_TRACE_LOCK_TYPE    my_lock_t
3 #define RX_TRACE_LOCK_INIT(lk) my_lock_init(&(lk))
4 #define RX_TRACE_LOCK_ACQUIRE(lk) my_lock_acquire(&(lk))
5 #define RX_TRACE_LOCK_RELEASE(lk) my_lock_release(&(lk))
6 #include "rxnet/trace.h"
```

Lectura adicional

- **Código fuente:** `c/` — runtime (~200 líneas), fsm (~150 líneas), pn (~200 líneas)
- **Tests:** `c/tests/` — `test_fsm.c`, `test_pn.c`, `parity_runner.c`
- **Ejemplos:** `c/examples/fsm/` y `c/examples/pn/` — ejecutables para macOS/Linux

- **Especificaciones:** [docs/specs/c/requirements.md](#) y [docs/specs/c/design.md](#)
- **Implementación Python:** [python/](#) — misma semántica, API ergonómica en Python 3.14+